

Kantenkontraktionsalgorithmen für das Multicut-Problem

Anton Drozd

Matrikelnummer: 4849660

Bachelor-Arbeit

Erstgutachter

Prof. Dr. rer. nat. Björn Andres

Zweitgutachter

Prof. Dr. rer. nat. Stefan Gumhold

Betreuer

M.Sc. Jannik Irmay

Eingereicht am: 25. August 2025

Inhaltsverzeichnis

1	Einleitung	4
2	Multicut-Problem	6
3	Kantenkontraktionsansätze für das Multicut-Problem	8
3.1	Greedy Additive Edge Contraction (GAEC)	8
3.2	Parallele Kantenkontraktion	9
3.2.1	Maximales Matching	12
3.2.2	Maximaler Spannwald ohne Konflikte	13
3.2.3	Reduktion der Kontraktionsmenge	16
4	Experimente	18
5	Fazit	25
	Literaturverzeichnis	26

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich das vorliegende Dokument mit dem Titel *Kantenkontraktionsalgorithmen für das Multicut-Problem* selbstständig und ohne unzulässige Hilfe Dritter verfasst habe. Es wurden keine anderen als die in diesem Dokument angegebenen Hilfsmittel und Quellen benutzt. Die wörtlichen und sinngemäß übernommenen Zitate habe ich als solche kenntlich gemacht. Es waren keine weiteren Personen an der geistigen Herstellung des vorliegenden Dokumentes beteiligt. Mir ist bekannt, dass die Nichteinhaltung dieser Erklärung zum nachträglichen Entzug des Hochschulabschlusses führen kann.

Dresden, 25. August 2025

A handwritten signature in black ink, appearing to be 'Anton Drozd', written in a cursive style.

Anton Drozd

1 Einleitung

Die Dekomposition eines Graphen in bedeutungsvolle Cluster stellt ein fundamentales Problem der kombinatorischen Optimierung dar. Das Multicut-Problem (Chopra & Rao, 1993) bietet einen populären Ansatz, einen Graphen basierend auf Affinitäten zwischen Knoten in Cluster beliebiger Anzahl und Größe zu zerlegen. Diese Eigenschaften der Cluster müssen bei der Definition einer Probleminstanz nicht bekannt sein, sondern ergeben sich aus den zulässigen Lösungen. Dies ist besonders vorteilhaft in Anwendungen, bei denen Anzahl und Größe der Cluster aus den Daten geschätzt werden soll, wie zum Beispiel in der Bildsegmentierung oder der Analyse sozialer Netzwerke. Das Multicut-Problem und seine Erweiterung wie der Higher-Order-Multicut (Kim et al., 2011) oder Lifted-Multicut (Keuper et al., 2015) haben zahlreiche Anwendungen in den Bereichen des maschinellen Lernens, Computer Vision, biomedizinische Bildanalyse, Data Mining und darüber hinaus gefunden. Beispiele hierfür sind unüberwachte Bildsegmentierung (Andres et al., 2011), Zell-Tracking (Jug et al., 2016), instanztrennende semantische Segmentierung (Kirillov et al., 2017), Bewegungssegmentierung (Keuper et al., 2018), Konnektomik (Andres et al., 2012) sowie zahlreiche weitere Anwendungsbereiche.

Erstmals wurde das Problem gegen Ende des 20. Jahrhunderts untersucht. Zunächst wurde es von Grötschel und Wakabayashi (1989) für vollständige Graphen betrachtet und später von Chopra und Rao (1993) auf allgemeine Graphen verallgemeinert. Da das Multicut-Problem und seine Erweiterungen NP-schwer sind, existiert mit hoher Wahrscheinlichkeit kein effizienter Lösungsalgorithmus, der das Problem exakt löst. Aus diesem Grund kommen in der Praxis vor allem heuristische und approximative Verfahren zum Einsatz. Von besonderer Bedeutung sind lokale Suchverfahren, die auf Kantenkontraktionen basieren. Dabei werden typischerweise Kanten mit hohen positiven Kosten kontrahiert, da diese darauf hindeuten, dass ihre Endknoten demselben Cluster angehören. Dieser Prozess wird solange wiederholt, bis keine weiteren Kanten als Kontraktionskandidaten identifiziert werden können.

Der *Greedy-Additive-Edge-Contraction*-Algorithmus (GAEC) (Keuper et al., 2015) ist ein effizienter lokaler Suchalgorithmus, der ausschließlich Kantenkontraktionen als Suchoperation verwendet. In jeder Iteration kontrahiert GAEC eine Kante mit höchsten positiven Kosten, bis keine solche Kante mehr existiert.

Abbas und Swoboda (2022) stellten kürzlich eine Variante des GAEC-Algorithmus vor, bei der in jeder Iteration mehrere Kanten gleichzeitig kontrahiert werden. Dies ermöglicht ei-

1 Einleitung

nerseits eine parallele Ausführung zur Beschleunigung der Laufzeit und kann andererseits dazu beitragen, ungünstige lokale Optima zu vermeiden. Die Wahl der Kontraktionsmenge $S \subseteq E$ für einen gegebenen Graphen $G = (V, E)$ stellt dabei einen zentralen Schritt dar. Abbas und Swoboda (2022) schlagen hierzu zwei Heuristiken vor: ein maximales Matching sowie einen maximalen konfliktfreien Spannwald. Zwar werden in ihrer Arbeit auch die Zielfunktionswerte betrachtet, jedoch richtet sich der primäre Fokus auf die Beschleunigung der Laufzeit, insbesondere bei sehr großen Instanzen des Multicut-Problems.

Die vorliegende Arbeit hingegen konzentriert sich auf die Analyse der Zielfunktionswerte, die beide Heuristiken im Vergleich zum GAEC-Algorithmus erzielen. Zusätzlich wird ein Hyperparameter $\tau \in [0, 1]$ eingeführt, um die maximal zulässige Kardinalität der Kontraktionsmenge S zu regulieren und dessen Einfluss auf die Zielfunktion zu untersuchen. Für $\tau = 0$ entsprechen beide Heuristiken dem klassischen GAEC-Algorithmus, bei dem pro Iteration genau eine Kante mit maximalen positiven Kosten kontrahiert wird, während mit wachsendem τ zunehmend mehr Kanten der jeweiligen Heuristik parallel kontrahiert werden.

Für die betrachteten Datensätze zeigt sich ein Trend zunehmender Lösungsqualität bei fallendem τ . Die erzielten Resultate erreichen dabei die Qualität des GAEC-Algorithmus und übertreffen diese in mehreren Fällen. Alle in dieser Arbeit verwendeten Implementierungen sind unter <https://github.com/yroja/Kantenkontraktionsalgorithmen> zugänglich.

2 Multicut-Problem

Das Multicut-Problem ist ein kombinatorisches Optimierungsproblem, das sich mit der Dekomposition (Clustering) von Graphen beschäftigt. Dabei wird ein Graph in Komponenten (Cluster) zerlegt, ohne die Anzahl, Größe oder andere Eigenschaften dieser Komponenten zu kennen. Dies verallgemeinert das Problem der Partitionierung einer Menge und für vollständige Graphen spezialisiert es sich zu diesem. Zur formalen Darstellung wird in diesem Kapitel die Terminologie von Horňáková et al. (2017) verwendet.

Definition 1. (Horňáková et al., 2017) Sei $G = (V, E)$ ein beliebiger Graph. Ein Subgraph $G' = (V', E')$ ist eine Komponente von G genau dann, wenn G' nicht leer, knoteninduziert und zusammenhängend ist. Eine Partition Π ist eine Dekomposition von G genau dann, wenn für jedes $U \in \Pi$ der durch U in G induzierte Subgraph $(U, E \cap \binom{U}{2})$ zusammenhängend (und somit eine Komponente von G) ist.

Für jeden Graphen G bezeichnen wir mit $D_G \subset 2^{2^V}$ die Menge aller Dekompositionen von G .

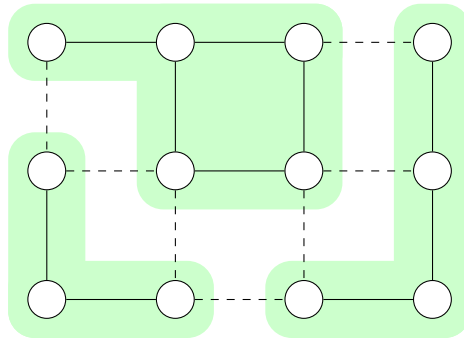


Abbildung 2.1: Nach Horňáková et al. (2017). Eine Dekomposition eines Graphen ist eine Partition der Knotenmenge in zusammenhängende Teilmengen. Diese wird durch die Menge der Kanten charakterisiert (hier als gestrichelte Linien dargestellt), die verschiedene Komponenten miteinander verbinden. Diese Kantenmenge ist der Multicut des Graphen.

2 Multicut-Problem

Wir untersuchen die Menge aller Dekompositionen eines Graphen anhand ihrer Charakterisierung als Menge von Multicuts.

Definition 2. Für jeden Graphen $G = (V, E)$ ist die Teilmenge $M \subseteq E$ ein Multicut von G genau dann, wenn für jeden Zyklus $C \subseteq E$ von G gilt: $|C \cap M| \neq 1$

Lemma 1. (Chopra & Rao, 1993) Es genügt, in Definition 2, ausschließlich sehnenfreie Zyklen zu berücksichtigen.

Für jeden Graphen G bezeichnen wir mit $M_G \subseteq 2^E$ die Menge aller Multicuts von G . Zu jeder Dekomposition eines Graphen G gehört die Menge jener Kanten, die zwischen den Komponenten verlaufen. Diese Menge ist ein Multicut von G und wird von der Dekomposition induziert. Tatsächlich ist die Abbildung von Dekompositionen zu den induzierten Multicuts eine Bijektion von D_G nach M_G (Horňáková et al., 2017). Abbildung 2.1 illustriert dies exemplarisch. Die charakteristische Funktion $y : E \rightarrow \{0, 1\}$ eines Multicuts $y^{-1}(1)$ legt für jede Kante $\{u, v\} = e \in E$ fest, ob die anliegenden Knoten zur gleichen Komponente ($y_e = 0$) oder zu verschiedenen Komponenten ($y_e = 1$) gehören. Laut Definition eines Multicuts sind diese Entscheidungen jedoch nicht notwendigerweise unabhängig voneinander.

Lemma 2. (Chopra & Rao, 1993) Für jeden Graphen $G = (V, E)$ und jede Abbildung $y : E \rightarrow \{0, 1\}$ ist die Menge $y^{-1}(1)$ derjenigen Kanten die auf 1 abgebildet werden, genau dann ein Multicut von G , wenn die folgenden Ungleichungen erfüllt sind:

$$\forall C \in \text{cycles}(G) \forall e \in C : y_e \leq \sum_{e' \in C \setminus \{e\}} y_{e'} \quad (2.1)$$

Da wir nun eine endliche Menge E , Entscheidungen $y : E \rightarrow \{0, 1\}$ und die Nebenbedingung 2.1 haben, kann die Menge der zulässigen Lösungen für das Multicut-Problem definiert werden:

$$\mathcal{Y} = \left\{ y : E \rightarrow \{0, 1\} \mid \forall C \in \text{cycles}(G) \forall e \in C : y_e \leq \sum_{e' \in C \setminus \{e\}} y_{e'} \right\} \quad (2.2)$$

Das Ziel ist es, den gegebenen Graphen in bedeutungsvolle Cluster zu unterteilen, sodass die Summe der Kantenkosten minimal ist.

Definition 3. Sei $G = (V, E)$ ein ungerichteter Graph und $c : E \rightarrow \mathbb{R}$ die entsprechende Kostenfunktion. Das Multicut-Problem besteht darin, das folgende Optimierungsproblem zu lösen:

$$\min_{y \in \mathcal{Y}} \sum_{e \in E} c_e y_e \quad (2.3)$$

Das Multicut-Problem ist eng verwandt mit dem *Correlation Clustering*. Beide Probleme wurden intensiv untersucht, insbesondere von Chopra und Rao (1993), Bansal et al. (2004) und Demaine et al. (2006). Bansal et al. (2004) zeigen die NP-Schwere des *Correlation Clusterings* durch eine Reduktion vom *k-Terminal-Cut-Problem*, dessen NP-Schwere ein zentrales Ergebnis von Dahlhaus et al. (1994) ist. Aufgrund der polynomiellen Äquivalenz zwischen dem Multicut-Problem und dem *Correlation Clustering*, vgl. Andres et al. (2023), folgt insbesondere, dass auch das Multicut-Problem NP-schwer ist.

3 Kantenkontraktionsansätze für das Multicut-Problem

Das Multicut-Problem und seine Erweiterungen sind NP-schwer, sodass es als unwahrscheinlich gilt, jemals einen effizienten Algorithmus zu finden, der das Problem exakt löst. Da in der Praxis häufig große Probleminstanzen auftreten, wurden leistungsfähige heuristische und approximative Verfahren entwickelt. Besonders relevant sind dabei schnelle lokale Suchalgorithmen, bei denen Kantenkontraktionen als weit verbreitete Suchoperation gelten. Diese Kantenkontraktionsalgorithmen basieren auf der iterativen Auswahl von Kanten mit hohen positiven Kosten, da diese darauf hinweisen, dass die jeweils inzidenten Knoten derselben Komponente angehören. Entsprechend ausgewählte Kanten werden kontrahiert, sodass ihre Endpunkte in einer gemeinsamen Komponente zusammengeführt werden. Dieser Vorgang wird so lange wiederholt, bis keine weiteren Kanten für eine Kontraktion identifiziert werden können.

3.1 Greedy Additive Edge Contraction (GAEC)

Der von Keuper et al. (2015) vorgestellte Greedy-Additive-Edge-Contraction-Algorithmus (GAEC) ist ein effizienter lokaler Suchalgorithmus, der ausschließlich Kantenkontraktionen als lokale Suchoperation verwendet. Bei diesem Verfahren können die Komponenten nur wachsen, und die Anzahl der Komponenten verringert sich in jedem Schritt um exakt eins. Daher beginnt der Algorithmus mit der feinsten Dekomposition Π_0 von $G = (V, E)$, die aus einelementigen Komponenten besteht.

Algorithmus 1 erhält als Eingabe eine Instanz des Multicut-Problems (Definition 3), bestehend aus einem ungerichteten Graphen $G = (V, E)$ sowie einer Kostenfunktion $c : E \rightarrow \mathbb{R}$. Als Ausgabe wird eine Dekomposition des Graphen G erzeugt. Die Darstellung der aktuellen Dekomposition erfolgt über den Graphen $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, wobei jeder Knoten $a \in \mathcal{V}$ einer Komponente in G entspricht und eine Kante $ab \in \mathcal{E}$ genau dann existiert, wenn die beiden Komponenten in G benachbart sind. In jeder Iteration vereinigt der Algorithmus das benachbarte Komponentenpaar, dessen Zusammenführung den Zielfunktionswert am stärksten senkt. Der Algorithmus terminiert, sobald keine Kontraktion den Zielfunktionswert strikt verringern würde.

Algorithmus 1 : Greedy-Additive-Edge-Contraction (GAEC)

Eingabe : Graph $G = (V, E)$,

Kostenfunktion $c : E \rightarrow \mathbb{R}$

Ausgabe : Dekomposition $\mathcal{G} = (\mathcal{V}, \mathcal{E})$

```

1 while  $\mathcal{E} \neq \emptyset$  do
2    $ab := \operatorname{argmax}_{a'b' \in \mathcal{E}} \mathcal{X}_{a'b'}$ 
3   if  $\mathcal{X}_{ab} < 0$  then
4     break
5   contract  $ab$  in  $\mathcal{G}$ 
6   foreach  $ab \neq ab' \in \mathcal{E}$  do
7      $\mathcal{X}_{ab'} := \mathcal{X}_{ab'} + \mathcal{X}_{bb'}$ 

```

Implementierung Bei der Implementierung wird der Graph $\mathcal{G}$ durch eine Adjazenzliste realisiert. Eine Disjoint-Set-Datenstruktur partitioniert die Knotenmenge V , und eine Prioritätswarteschlange verwaltet die zulässigen Kontraktionskandidaten, indem sie diese gemäß ihrer Kosten $\mathcal{X} : \mathcal{E} \rightarrow \mathbb{R}$ sortiert.

Die Worst-Case-Laufzeit beträgt $\mathcal{O}(|V|^2 \log |V|)$ und ergibt sich daraus, dass in höchstens $|V|$ Iterationen jeweils bis zu $\deg G' \leq |V|$ Kanten entfernt werden, wobei das Entfernen einer einzelnen Kante $\mathcal{O}(\log \deg G') \in \mathcal{O}(\log |V|)$ Zeit erfordert.

3.2 Parallele Kantenkontraktion

Kürzlich stellten Abbas und Swoboda (2022) eine Variante des GAEC-Algorithmus vor, bei der in jeder Iteration mehrere Kanten gleichzeitig kontrahiert werden. Das hat zwei Vorteile: Erstens können mehrere Kontraktionen parallel ausgeführt werden, was zu einer schnelleren Laufzeit führt. Zweitens können ungünstige lokale Optima, die der GAEC-Algorithmus möglicherweise findet, vermieden werden. Ähnlich wie beim GAEC-Algorithmus hängt das Finden einer Lösung davon ab, Kanten zu kontrahieren, die mit hoher Wahrscheinlichkeit in derselben Komponente liegen. Zu diesem Zweck schlagen die Autoren einen Ansatz aus der linearen Algebra vor, bei dem Kantenkontraktionen als Sparse-Matrix-Matrix-Multiplikationen ausgedrückt werden. Diese Wahl begründet sich dadurch, dass das Verfahren speziell auf eine GPU-Implementierung ausgelegt ist, bei der durch die Nutzung paralleler primitiver Operationen eine signifikante Beschleunigung erzielt werden kann. Obwohl die Implementierungen in dieser Arbeit ausschließlich CPU-basiert sind, wird im Folgenden die Matrixdarstellung übernommen. Die nachstehenden Ausführungen folgen weitgehend der Darstellung, Terminologie und Argumentationsstruktur von Abbas und Swoboda (2022).

3 Kantenkontraktionsansätze für das Multicut-Problem

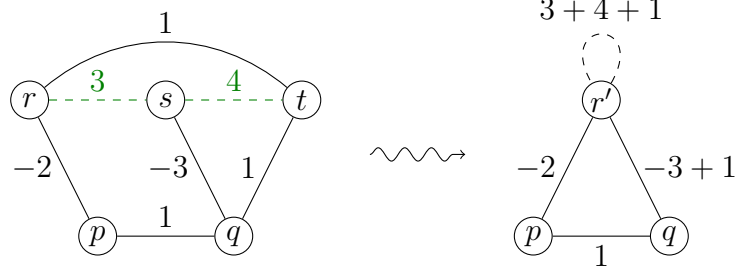


Abbildung 3.1: Nach Abbas und Swoboda (2022). Kontraktion eines Graphen mit der Kontraktionsmenge $S = \{rs, st\}$, wobei die Knoten r, s und t zu einem Cluster r' zusammengeführt werden. Die entsprechende Kontraktionsabbildung ist $f(p) = p, f(q) = q, f(r) = f(s) = f(t) = r'$. Nach der Kontraktion sind die Kanten qs und qt parallel, weshalb ihre Kosten addiert werden. Zudem weist der kontrahierte Graph eine Schleife auf, deren Kosten die Intra-Cluster-Ähnlichkeit repräsentieren.

Lemma 3. (Abbas & Swoboda, 2022) Gegeben sei ein ungerichteter Graph $G = (V, E)$ mit einer Kostenfunktion $c : E \rightarrow \mathbb{R}$ sowie eine Teilmenge $S \subseteq E$ der zu kontrahierenden Kanten. Es bezeichne $G' = (V', E')$ den Graphen, der durch Kontraktion der Kanten S entsteht, und $c' : E' \rightarrow \mathbb{R}$ die entsprechende (induzierte) Kostenfunktion.

(a) Die zugehörige surjektive Kontraktionsabbildung $f : V \rightarrow V'$, die die Knotenmenge V auf die kontrahierte Knotenmenge V' abbildet, ist bis auf die Isomorphie eindeutig definiert durch $f(u) = f(v) \Leftrightarrow \exists uv\text{-path}(V, S)$. Die kontrahierte Kantenmenge ist gegeben durch $E' = \{f(u)f(v) : f(u) \neq f(v), uv \in E\}$.

(b) Die Gewichte der kontrahierten Kanten sind $c'_{ij} = \sum_{uv \in E: f(u)=i, f(v)=j} c_{uv}, \forall ij \in E'$.

Lemma 3(a) beschreibt einen Zusammenhang zwischen der Kontraktionsabbildung f und der Kontraktionsmenge S . Gilt $f(u) = f(v)$ für zwei Knoten $u, v \in V$, so existiert ein Pfad zwischen ihnen im Teilgraphen $G = (V, S)$. Kanten, deren Endpunkte nicht kontrahiert werden, bleiben in E' erhalten. Lemma 3(b) beschreibt darüber hinaus, wie sich die Kosten der kontrahierten Kanten berechnen lassen: Sie ergeben sich durch das Aufsummieren der Kosten paralleler Kanten zwischen den jeweiligen Komponenten. Eine Veranschaulichung von Lemma 3 findet sich in Abbildung 3.1.

Definition 4. Gegeben sei ein ungerichteter Graph $G = (V, E)$ und eine Kostenfunktion $c : E \rightarrow \mathbb{R}$. Die (symmetrische) Adjazenzmatrix $A \in \mathbb{R}^{V \times V}$ ist definiert durch

$$A_{uv} = \begin{cases} c_{uv}, & \text{falls } uv \in E \\ 0, & \text{sonst} \end{cases}$$

3 Kantenkontraktionsansätze für das Multicut-Problem

Die Kantenkontraktion wird mithilfe einer Kantenkontraktionsmatrix durchgeführt, die wie folgt definiert ist:

Definition 5. (Abbas & Swoboda, 2022) Gegeben sei ein ungerichteter Graph $G = (V, E)$, eine zu kontrahierende Kantenmenge $S \subseteq E$, sowie die Kontraktionsabbildung f und die Menge der kontrahierten Knoten V' . Die Kantenkontraktionsmatrix $K_S \in \{0, 1\}^{V \times V'}$ ist definiert durch

$$(K_S)_{uu'} = \begin{cases} 1, & \text{falls } f(u) = u' \\ 0, & \text{sonst} \end{cases}$$

Lemma 4. (Abbas & Swoboda, 2022) Gegeben sei ein ungerichteter Graph $G = (V, E)$ mit einer Kostenfunktion $c : E \rightarrow \mathbb{R}$, eine zu kontrahierende Kantenmenge $S \subseteq E$ sowie eine entsprechende Kontraktionsabbildung f .

- (a) Die Adjazenzmatrix des kontrahierten Graphen entspricht $K_S^\top A K_S - \text{diag}(K_S^\top A K_S)$, wobei $\text{diag}(\cdot)$ die Diagonale einer Matrix bezeichnet.
- (b) Für das Diagonalelement gilt: $(K_S^\top A K_S)_{u'u'} = \sum_{uv \in E: u'=f(u)=f(v)} c_{uv}$.

Lemma 4(a) beschreibt eine Möglichkeit, den kontrahierten Graphen parallel mittels Sparse-Matrix-Matrix-Multiplikation zu berechnen. Lemma 4(b) erlaubt eine effiziente Bewertung, ob die neu gebildeten Cluster den Zielfunktionswert verringern. Insbesondere gilt: Enthält die Diagonale ausschließlich positive Einträge, so verringert sich der Zielfunktionswert infolge der Kontraktion ebenfalls.

Algorithmus 2 : Parallele Kantenkontraktion

Eingabe : Graph $G = (V, E)$,
Kostenfunktion $c : E \rightarrow \mathbb{R}$
Ausgabe : Kontrahierter Graph $G' = (V', E')$,
Kostenfunktion $c' : E' \rightarrow \mathbb{R}$
Kontraktionsabbildung $f : V \rightarrow V'$

- 1 Berechne die Kontraktionsmenge $S \subseteq E$
 - 2 Berechne die Adjazenzmatrix A von G
 - 3 Konstruiere die Kontraktionsabbildung $f : V \rightarrow V'$
 - 4 Konstruiere die Kantenkontraktionsmatrix K_S
 - 5 $A' = K_S^\top A K_S - \text{diag}(K_S^\top A K_S)$
 - 6 Berechne den kontrahierten Graph $G' = (V', E')$ und die Kostenfunktion $c' : E' \rightarrow \mathbb{R}$ aus A'
-

Algorithmus 2 zeigt eine Aktualisierungsiteration, die Kantenkontraktionen gemäß Lemma 4(a) durchführt. Der Algorithmus erhält als Eingabe eine Instanz des Multicut-Problems (Definition 3) und gibt den kontrahierten Graphen $G' = (V', E')$, die entsprechende Kostenfunktion $c' : E' \rightarrow \mathbb{R}$ sowie der Kontraktionsabbildung $f : V \rightarrow V'$ zurück.

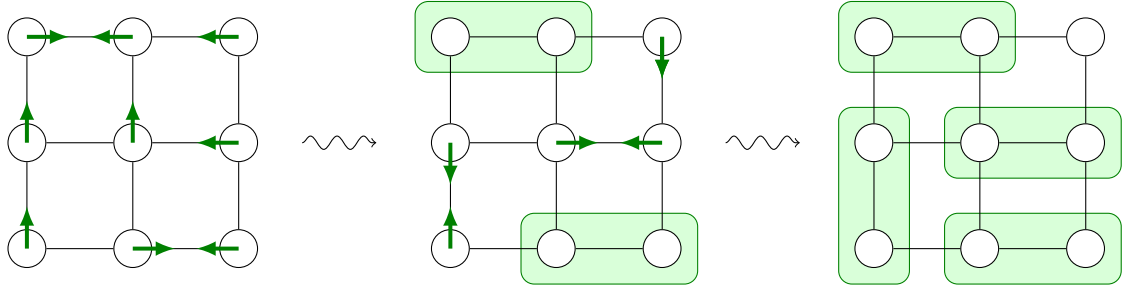


Abbildung 3.2: Vorgehensweise der *One-Phase-Handshaking-Variante* auf dem Graphen G ; innerhalb von zwei Iterationen wird ein *maximales* Matching gefunden (im gezeigten Beispiel sogar ein *maximum* Matching).

Ein zentraler Schritt, um eine gute Aktualisierung sicherzustellen, ist die Wahl der Kontraktionsmenge S in Zeile 1. Ziel ist es, Kanten behutsam zu wählen, um fehlerhafte Kontraktionen zu vermeiden. Abbas und Swoboda (2022) schlagen zu diesem Zweck zwei Heuristiken vor: ein maximales Matching sowie einen maximalen, konfliktfreien Spannwald. Beide Heuristiken ziehen hierbei ausschließlich positive Kanten in Betracht, sodass die erzeugte Kontraktionsmenge S keine negativen Kanten enthält. Zudem stellen diese Strategien sicher, dass die resultierende Kontraktion den Zielfunktionswert verringert. Im von Abbas und Swoboda (2022) vorgestellten primalen Algorithmus wird zunächst die Kontraktionsmenge S durch ein maximales Matching bestimmt. Sobald jedoch nicht mehr genügend Kanten gefunden werden (d.h. weniger als $0.1|V|$), erfolgt ein Wechsel zum Spannwald-basierten Ansatz. In dieser Arbeit wird darüber hinaus eine Variante betrachtet, die ausschließlich den Spannwald-basierten Ansatz nutzt.

3.2.1 Maximales Matching

Definition 6. Sei $G = (V, E)$ ein ungerichteter Graph.

- (a) Ein Matching $M \subseteq E$ von G ist eine Menge von Kanten, sodass kein Knoten $v \in V$ zu mehr als einer Kante in M inzident ist.
- (b) Ein maximales Matching $M \subseteq E$ von G ist ein Matching, das in keinem anderen Matching $M' \subseteq E$ echt enthalten ist.
Formal gilt: $M \not\subseteq M'$ für jedes andere Matching M' von G .
- (c) Ein maximum Matching $M \subseteq E$ von G ist ein Matching mit maximaler Kardinalität. Formal gilt: Für jedes Matching $M' \subseteq E$ ist $|M| \geq |M'|$.

Für die effiziente Berechnung eines maximalen Matchings schlagen Abbas und Swoboda (2022) die Nutzung einer GPU-basierten Variante des Luby-Jones-Handshaking-Algorithmus vor, wie er von Cohen und Castonguay (2012) beschrieben wird. Dieser Algorithmus wird auf den positiven Kanten des Graphen ausgeführt, wobei die so ermittelten Knotenpaare anschließend als Kontraktionsmenge S gewählt werden.

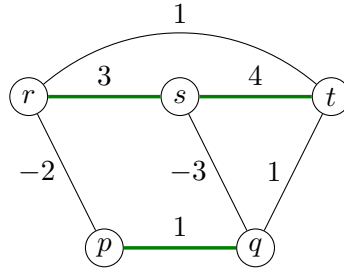


Abbildung 3.3: Beispiel für einen maximalen konfliktfreien Spannwald $F = (V, E')$ (grün hervorgehoben) in einem gegebenen ungerichteten Graphen $G = (V, E)$ mit $E' = \{rs, st, pq\}$. Beachte, dass die Kante tq nicht in E' enthalten ist, da sonst durch die negativen Kanten rp und sq Konflikte entstehen würden.

In dieser Arbeit kommt die sogenannte *One-Phase-Handshaking*-Variante zum Einsatz (siehe Abbildung 3.2). Dabei reicht jeder Knoten seine „Hand“ zu dem Nachbarn mit der höchsten Kantengewichtung. Reichen zwei Knoten einander die „Hand“, stellt dies einen Handschlag dar, und die beiden Knoten bilden ein Paar, das Teil des berechneten Matchings wird. Dieser Vorgang wird iterativ wiederholt, bis keine weiteren Paare mehr gebildet werden können.

Dieses Verfahren beschreibt einen Greedy-Algorithmus, der in jeder Iteration nur Kanten zwischen Knoten betrachtet, die noch nicht im Matching enthalten sind. Für jeden dieser Knoten wird die Kante mit dem höchsten Gewicht zu einem benachbarten Knoten ausgewählt, der ebenfalls noch nicht Teil des Matchings ist. Die Auswahl erfolgt ohne Berücksichtigung einer globalen Optimierung. Diese Greedy-Strategie garantiert, dass das gefundene Matching *maximal*, jedoch nicht notwendigerweise ein *maximum* Matching darstellt.

3.2.2 Maximaler Spannwald ohne Konflikte

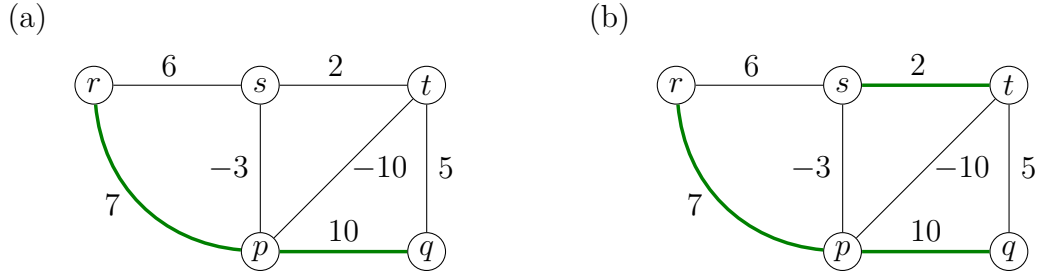
Definition 7. Sei $G = (V, E)$ ein zusammenhängender, ungerichteter Graph. Ein Spannbaum von G ist ein Teilgraph $T = (V, E')$ mit $E' \subseteq E$, der alle Knoten V enthält und ein Baum ist, das heißt, er ist zusammenhängend und azyklisch.

Definition 8. Sei $G = (V, E)$ ein ungerichteter Graph. Ein Spannwald von G ist ein Teilgraph $F = (V, E')$ mit $E' \subseteq E$, der alle Knoten V enthält und dessen zusammenhängende Komponenten jeweils Spannbäume sind.

Definition 9. Sei $G = (V, E)$ ein ungerichteter Graph und $c: E \rightarrow \mathbb{R}$ eine Kostenfunktion. Ein maximaler Spannwald $F = (V, E')$ von G ist ein Spannwald, für den die Summe der Kantenkosten

$$\sum_{e \in E'} c_e$$

unter allen möglichen Spannwäldern von G maximal ist.



Abbildungung 3.4: Vergleich zweier konfliktfreier Spannwürder (grün hervorgehoben) für den gegebenen Graphen $G = (V, E)$. In (a) ist ein konfliktfreier Spannwald dargestellt, der aus der von Abbas und Swoboda (2022) vorgeschlagenen Strategie resultiert. Zunächst wird ein maximaler Spannwald F berechnet, der die Kanten $\{pq, rp, rs, tq\} \subseteq E$ umfasst. Anschließend werden Konflikte durch das Entfernen der Kanten rs und tq aus dem Spannwald gelöst. Der resultierende Spannwald ist konfliktfrei, aber nicht maximal für den gegebenen Graphen. In (b) wird der maximale, konfliktfreie Spannwald F dargestellt, der die Kanten $\{pq, rp, st\} \subseteq E$ enthält.

Definition 10. Sei $G = (V, E)$ ein ungerichteter Graph und $c : E \rightarrow \mathbb{R}$ eine Kostenfunktion. Ein Spannwald $F = (V, E')$ heißt konfliktfrei, wenn für alle Kanten $uv \in E$ mit $c_{uv} < 0$ gilt, dass u und v nicht durch einen Pfad in F verbunden sind.

Abbas und Swoboda (2022) schlagen zur Berechnung eines maximalen, konfliktfreien Spannwaldes eine Strategie vor, bei der zunächst mit einer GPU-Version des Borůvka-Algorithmus (Hwu, 2011) ein maximaler Spannwald konstruiert wird. Anschließend werden Konflikte gelöst, indem über alle negativen Kanten ij iteriert und, falls ein eindeutiger Pfad zwischen i und j in dem Spannwald existiert, die Kante mit den geringsten positiven Kosten auf diesem Pfad entfernt wird.

Zur Berechnung des maximalen Spannwaldes wird in dieser Arbeit der Kruskal-Algorithmus (Kruskal, 1956) verwendet. Dabei handelt es sich um einen Greedy-Algorithmus, der zunächst die Kanten entsprechend ihrer Kosten in absteigender Reihenfolge sortiert. Die Greedy-Strategie besteht darin, die Kanten gemäß der sortierten Reihenfolge in den Spannwald aufzunehmen. Um die azyklische Eigenschaft der Spannbäume zu bewahren, werden die Knoten durch eine Disjoint-Set-Datenstruktur verwaltet. Eine Kante wird nur dann aufgenommen, falls die anliegenden Knoten in unterschiedlichen Partitionen liegen.

Da in dieser Arbeit bereits für den initialen Graphen ein maximaler, konfliktfreier Spannwald zur Berechnung von S verwendet wird, werden Konflikte bereits während der Konstruktion des Spannwaldes verhindert. Somit entfällt die aufwendige Berechnung aller Pfade zwischen den Endpunkten negativer Kanten. Zu diesem Zweck wird der Kruskal-Algorithmus um ein Konzept des Mutex-Watershed-Algorithmus (Wolf et al., 2021) erweitert. Dabei wird eine Mutex-Datenstruktur eingeführt, die negative Kanten erfasst, deren Endpunkte in unterschiedlichen Komponenten liegen. Diese Datenstruktur entspricht einer Adjazenzliste, die für jede Komponente die Nachbarn speichert, zu de-

3 Kantenkontraktionsansätze für das Multicut-Problem

nen im aktuellen Graphen eine negative Kante vorliegt. Eine Kante wird nur dann in den Spannwald aufgenommen, wenn sowohl die Endpunkte in unterschiedlichen Partitionen der genannten Disjoint-Set-Datenstruktur liegen als auch kein Eintrag in der Mutex-Datenstruktur zwischen den entsprechenden Komponenten existiert. In diesem Fall werden sowohl die Disjoint-Set- als auch die Mutex-Datenstruktur aktualisiert. Abbildung 3.3 zeigt exemplarisch einen maximalen, konfliktfreien Spannwald für einen gegebenen Graphen $G = (V, E)$.

Algorithmus 3 : Mutex-Kruskal-Algorithmus

Eingabe : Positive Kanten E^+ ,
 Mutex-Datenstruktur *mutexes*
Ausgabe : Konfliktfreier Spannwald F

```

2 Initialize components and conflicts as disjoint-set data structures
3  $F \leftarrow \emptyset$ 
4 foreach  $uv \in E^+$  (sorted descending by cost) do
5    $root\_u \leftarrow components.find(u)$ 
6    $root\_v \leftarrow components.find(v)$ 
7   if  $root\_u \neq root\_v$  then
18      $components.merge(root\_u, root\_v)$ 
9      $ru \leftarrow conflicts.find(u)$ 
10     $rv \leftarrow conflicts.find(v)$ 
11    if  $ru \notin mutexes[rv]$  then
12       $F \leftarrow F \cup \{uv\}$ 
13       $conflicts.merge(ru, rv)$ 
14      Update mutexes
15    end
16  end
17 end
```

Algorithmus 3 erhält die folgenden zwei Eingaben: die Menge der positiven Kanten $E^+ := \{e \in E \mid c_e > 0\}$, sortiert in absteigender Reihenfolge ihrer Kosten, sowie eine initialisierte Mutex-Datenstruktur zur Verwaltung von Konflikten.

Abbas und Swoboda (2022) beschreiben eine Strategie, bei der zunächst ein maximaler Spannwald konstruiert und im Anschluss auftretende Konflikte gelöst werden. Der resultierende konfliktfreie Spannwald ist jedoch nicht mehr zwingend maximal (siehe Abbildung 3.4). Um diese Logik beizubehalten, nutzt der CPU-basierte Algorithmus 3 zwei Disjoint-Set-Datenstrukturen. *components* führt stets eine Join-Operation durch, wenn u und v in unterschiedlichen Komponenten liegen. Liegt zusätzlich kein Konflikt vor, wird zudem in *conflicts* eine Join-Operation ausgeführt, $uv \in E^+$ in den Spannwald F aufgenommen und die Mutex-Datenstruktur aktualisiert. Der so berechnete Spannwald F ist konfliktfrei und entspricht der von Abbas und Swoboda (2022) vorgestellten Logik.

Die Wahl des Kruskal-Algorithmus erweist sich hinsichtlich der Nutzung des Hyperparameters τ als sinnvoll, da die Kanten in absteigender Reihenfolge ihrer Kosten in den

Spannwald aufgenommen werden und Algorithmus 3 somit terminieren kann, sobald die maximal zulässige Anzahl der Kanten erreicht ist.

3.2.3 Reduktion der Kontraktionsmenge

In dieser Arbeit wird ein Hyperparameter $\tau \in [0, 1]$ eingeführt, der eine obere Schranke für die Kardinalität der Kontraktionsmenge S festlegt, um diese zu regulieren und dessen Auswirkungen auf die Zielfunktion zu untersuchen. Dieser Parameter wird als zusätzliches Argument an den jeweiligen Algorithmus zur Berechnung von S übergeben und stellt sicher, dass

$$|S| \leq \max(1, \lceil |V| \cdot \tau \rceil)$$

gilt. Diese Schranke wird einmalig für den initialen Graphen $G = (V, E)$ bestimmt und in allen nachfolgenden Aktualisierungsiterationen beibehalten. Eine Verringerung von τ reduziert die maximal zulässige Anzahl der Kanten in der Kontraktionsmenge und erhöht entsprechend die Anzahl der erforderlichen Aktualisierungsiterationen.

Ziel ist es, die Kontraktionsmenge S auf die $\max(1, \lceil |V| \cdot \tau \rceil)$ Kanten mit den höchsten Kosten des berechneten maximalen Matchings bzw. des konfliktfreien Spannwaldes zu beschränken. Da Algorithmus 3 die Kanten in absteigender Reihenfolge ihrer Kosten zum Spannwald $F = (V, E')$ hinzufügt, kann er terminieren, sobald $|E'|$ die obere Schranke erreicht, ohne den gesamten Spannwald berechnen zu müssen. Der *Luby-Jones-Handshaking*-Algorithmus fügt Kanten hingegen nicht in absteigender Reihenfolge ihrer Kosten zum Matching hinzu, sodass stets das gesamte Matching berechnet werden muss, unabhängig von τ . Die Kanten des Matchings werden anschließend nach absteigenden Kosten sortiert, und die ersten $\max(1, \lceil |V| \cdot \tau \rceil)$ Kanten zurückgegeben.

Aufgrund der azyklischen Eigenschaft eines Spannwaldes F kann dieser höchstens $|V| - 1$ Kanten enthalten, während ein maximales bzw. maximum Matching höchstens $\lfloor \frac{|V|}{2} \rfloor$ Kanten umfasst. Für $\tau = 1$ entspricht die Kontraktionsmenge S in beiden Heuristiken der vollständig vom jeweiligen Verfahren ermittelten Kantenmenge, gemäß Abbas und Swoboda (2022). Sowohl der *Mutex-Kruskal*-Algorithmus (Algorithmus 3) als auch die verwendete Implementierung des *Luby-Jones-Handshaking*-Algorithmus liefern die Kanten in absteigender Reihenfolge ihrer Kosten zurück. Insbesondere folgt daraus, dass Algorithmus 2 für $\tau = 0$ dem *GAEC*-Algorithmus (Algorithmus 1) entspricht. In diesem Fall wird jeweils nur eine Kante mit maximalen Kosten gewählt ($|S| = 1$).

Abbildung 3.5 illustriert die Vorgehensweise der drei Heuristiken für einen gegebenen Graphen bei $\tau = 1$. Dabei lassen sich Unterschiede hinsichtlich der Kardinalität der Kontraktionsmenge, der Anzahl der erforderlichen Iterationen sowie der erzielten Zielfunktionswerte beobachten. Für den betrachteten Graphen erreicht der GAEC-Algorithmus mit -4 den optimalen Zielfunktionswert.

3 Kantenkontraktionsansätze für das Multicut-Problem

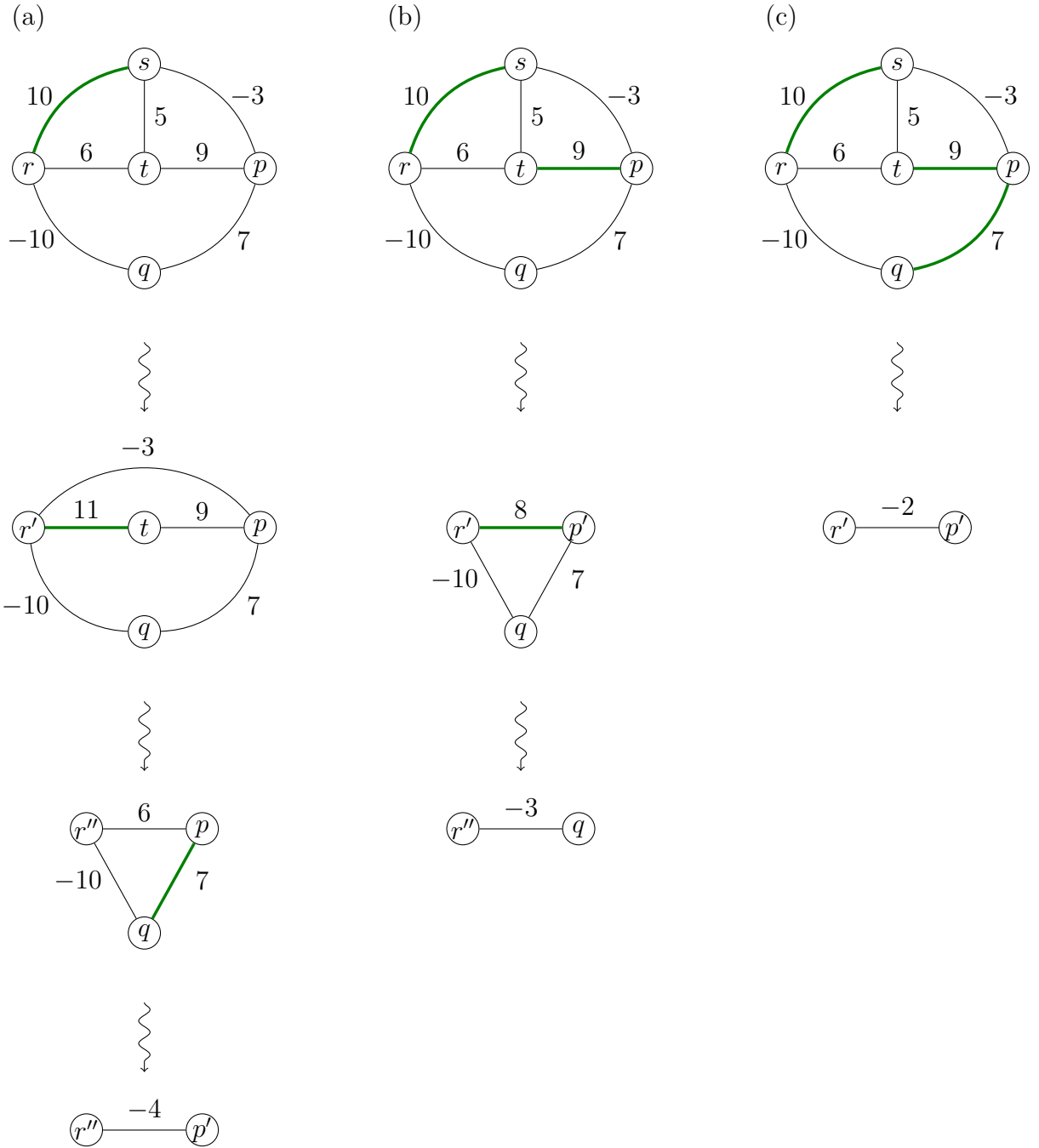


Abbildung 3.5: Vorgehensweise der drei Heuristiken für einen gegebenen Graphen $G = (V, E)$. In Subabbildung (a) wird die Funktionsweise des GAEC-Algorithmus (Algorithmus 1) dargestellt, während (b) bzw. (c) die parallele Kantenkontraktion (Algorithmus 2) in Kombination mit einem maximalen Matching (mittels *Luby-Jones-Handshaking-Algorithmus*) bzw. einem maximalen, konfliktfreien Spannwald (Algorithmus 3) illustriert.

4 Experimente

In diesem Kapitel werden die in den vorangegangenen Kapiteln beschriebenen Heuristiken für das Multicut-Problem anhand verschiedener Anwendungsfälle evaluiert, wie der Neuronensegmentierung eines Fruchtfliegengehirns im Kontext der Konnektomik (Pape et al., 2017) oder der unüberwachten Bildsegmentierung auf dem Cityscapes-Datensatz (Cordts et al., 2016). Die CPU-basierten Algorithmen wurden in C++ implementiert und auf einem System mit einem AMD FX-6350 Prozessor ausgeführt.

Datensätze Die Datensätze *Connectomics-SP* und *Cityscapes* sind über das Structured Prediction Problem Archive (Swoboda et al., 2022) zugänglich, während alle weiteren Datensätze aus CP-Lib (Sørensen & Letchford, 2024) stammen.

Connectomics-SP: Dieser Datensatz enthält Instanzen zur Neuronensegmentierung des Fruchtfliegengehirns, basierend auf Rohdaten der CREMI-Challenge, die ursprünglich von Zheng et al. (2018) erhoben wurden. Für die Konvertierung zu Multicut-Instanzen wurden die Daten gemäß Pape et al. (2017) vorverarbeitet, indem zunächst Superpixel generiert und dadurch die Problemgröße reduziert wurde. Die meisten Instanzen stellen unterschiedliche Teilprobleme eines übergeordneten Gesamtproblems dar. Insgesamt stehen drei kleine (mit etwa 400.000 bis 600.000 Kanten), drei mittlere (4 bis 5 Millionen Kanten) sowie fünf große Instanzen (zwischen 28 und 650 Millionen Kanten) zur Verfügung. Die großen Instanzen werden aufgrund des erheblichen Speicherbedarfs von der Evaluierung ausgeschlossen, da die erforderlichen Ressourcen die verfügbaren Hardwarekapazitäten überschreiten und eine Bearbeitung auf dem verwendeten System somit nicht möglich ist.

Cityscapes: Für die unüberwachte Bildsegmentierung kommen 59 hochauflösende Bilder (2048×1024 Pixel) aus dem Validierungsdatensatz (Cordts et al., 2016) zum Einsatz. Die Konvertierung in Multicut-Instanzen basiert auf Kantenaffinitäten, die mithilfe des Verfahrens von Abbas und Swoboda (2021) auf einem regulären Gittergraphen mit Konnektivität vom Grad 4 berechnet wurden. Zusätzlich wurde der Graph durch längerreichende, grob abgetastete Kanten erweitert. Jede Instanz enthält etwa 2 Millionen Knoten und rund 9 Millionen Kanten.

4 Experimente

ABR: Der ABR-Datensatz (englisch *Aggregation of Binary Relations*) umfasst Instanzen, die aus Objekten mit mehreren qualitativen Attributen generiert wurden. Für jedes Objektpaar wird eine Ähnlichkeitsbewertung vorgenommen, die auf der Anzahl übereinstimmender Attributwerte basiert. Fehlende Werte werden bei der Berechnung entsprechend berücksichtigt und führen zu einer Abschwächung des Ähnlichkeitsmaßes. Die so bestimmten Ähnlichkeitswerte werden als Kantengewichte in einem vollständigen, gewichteten Graphen interpretiert. Ziel ist es, Äquivalenzklassen zu identifizieren, in denen Objekte mit hoher Ähnlichkeit gruppiert werden. Die zugrunde liegenden ABR-Instanzen stammen aus unterschiedlichen Quellen. Der Großteil der Daten entstammt dem *UCI Machine Learning Repository* (Dua & Graff, 2019), ergänzt durch Instanzen aus der Fachliteratur, insbesondere den Arbeiten von Grötschel und Wakabayashi (1989). Da die Ähnlichkeitsbewertung auch Kanten mit Kosten 0 erzeugt, variieren die Instanzen mit 30 bis 797 Knoten trotz formaler Vollständigkeit in ihrer effektiven Kantenzahl zwischen 381 und 306.915.

MCF: Der MCF-Datensatz basiert auf dem *Machine Cell Formation Problem*, einem klassischen Optimierungsproblem der Produktionsplanung im Kontext der Group Technology. Eine Instanz besteht aus einer binären Matrix, deren Zeilen Maschinen und deren Spalten Bauteile repräsentieren. Einträge mit dem Wert 1 kennzeichnen, dass ein bestimmtes Bauteil von der zugehörigen Maschine bearbeitet wird. Zur Modellierung als *Clique Partitioning Problem* wird ein vollständiger Graph konstruiert, in dem Maschinen- und Bauteilknoten durch Kanten verbunden sind. Kanten zwischen zwei Maschinen oder zwei Bauteilen erhalten das Gewicht 0, während Kanten zwischen Maschinen und Bauteilen mit +1 (bei Bearbeitungsbeziehung) bzw. -1 (bei keiner Beziehung) gewichtet werden. Die zugrunde liegenden Instanzen stammen aus verschiedenen Veröffentlichungen und wurden unter anderem in den Arbeiten von Oosten et al. (2001) und Wang et al. (2006) behandelt. Dieser Datensatz umfasst Graphen mit 31 bis 1.150 Knoten und 220 bis 149.000 Kanten.

Cluster Editing: Dieser Datensatz basiert auf dem Cluster-Editing-Problem (auch bekannt als *Correlation Clustering*), einem Optimierungsproblem, das insbesondere im Bereich des Data Mining zur Identifikation zusammengehöriger Strukturen Anwendung findet. Ziel ist es, einen gegebenen Graphen durch eine minimale Anzahl von Kantenmodifikationen so zu verändern, dass eine Partition in Cliques entsteht. Die zugehörigen Instanzen umfassen ungerichtete Graphen mit variierenden Kantendichten. Die Daten wurden unter anderem gemäß dem Vorgehen von Simanchev et al. (2019) generiert, wobei verschiedene Graphgrößen und Kantendichten berücksichtigt werden, um verschiedene Schwierigkeitsgrade abzubilden. Der Datensatz umfasst Instanzen mit 50 bis 80 Knoten und 1.225 bis 3.160 Kanten.

Random: Dieser Datensatz enthält Instanzen ungerichteter, vollständiger Graphen, deren Kantengewichte größtenteils als gleichverteilte Zufallszahlen aus einem ganzzahligen Intervall $[l, u]$ gewählt wurden. Die Instanzen wurden unter anderem über Zhou et al. (2016) bezogen und von Sørensen und Letchford (2024) in Maximierungsinstanzen

4 Experimente

des *Clique-Partitioning-Problems* überführt. Zusätzlich umfasst der Datensatz Varianten mit normalverteilten Gewichten (z. B. $N(0, 5^2)$) sowie Instanzen, bei denen ein Teil der Kanten gezielt mit hohen negativen Werten belegt wurde. Einige Instanzen basieren darüber hinaus auf strukturierten Verfahren, etwa durch zufällige Bipartitionen, symmetrische Relationen oder Transformationen aus dem *Unconstrained Binary Quadratic Programming* (UBQP). Dieser Datensatz umfasst Instanzen mit 35 bis 2500 Knoten und entsprechend vollständiger Graphen $\frac{n(n-1)}{2}$ Kanten. Viele dieser Kanten haben jedoch Kosten von 0, sodass die effektive Kantenzahl deutlich geringer ist und von 590 bis 1.989.123 variiert.

Equicut: Dieser Datensatz bezieht sich auf das *Equicut-Problem*, welches dem *Clique-Partitioning-Problem* ähnelt, jedoch die zusätzliche Einschränkung enthält, dass genau zwei Cluster mit jeweils der Größe $\lfloor n/2 \rfloor$ bzw. $\lceil n/2 \rceil$ gebildet werden müssen, siehe etwa Brunetta et al. (1997). Durch Aufhebung dieser Größenbeschränkung lässt sich jede *Equicut*-Instanz in eine Instanz des *Clique-Partitioning-Problems* überführen. Damit die resultierende *Clique-Partitioning*-Instanz nicht trivial wird, ist es notwendig, dass sowohl positive als auch negative Kantengewichte vorliegen. Einige der von Brunetta et al. (1997) beschriebenen *Equicut*-Instanzen erfüllen diese Voraussetzung und wurden mithilfe eines Dichte-Parameters erzeugt. Bei einer Dichte von $k\%$ wird jede Kante mit einer Wahrscheinlichkeit von $(100 - k)\%$ auf 0 gesetzt, ansonsten wird das Kantengewicht mit einer Wahrscheinlichkeit von $k\%$ zufällig und gleichverteilt aus der Menge $\{-9, -8, \dots, -1\} \cup \{1, 2, \dots, 9\}$ gewählt. Diese Instanzen umfassen Graphen mit 50 bis 70 Knoten und weisen je nach Dichte (20 bis 100%) zwischen 142 und 1.225 Kanten auf.

Correlation: Dieser Datensatz basiert auf Instanzen des *Clique-Partitioning-Problems*, die durch die Berechnung von Korrelationen zwischen n unabhängigen Zufallsvariablen generiert wurden. Dabei werden die paarweise Korrelationskoeffizienten der Variablen berechnet und als Kantengewichte im Graphen verwendet. Durch dieses Vorgehen entstehen herausfordernde Instanzen, die sich gut für die Evaluierung von Algorithmen eignen. Der Datensatz umfasst insgesamt zehn Instanzen mit 40 bis 80 Knoten und 756 bis 3.066 Kanten.

Algorithmen Zur experimentellen Evaluation dienen die zuvor beschriebenen Algorithmen.

GAEC: Der *Greedy-Additive-Edge-Contraction*-Algorithmus (Algorithmus 1) kontrahiert in jeder Iteration eine Kante mit maximalen positiven Kosten. Hierbei wird die von Keuper et al. (2015) bereitgestellte CPU-Implementierung verwendet.

PECM: Diese Methode beschreibt eine parallele Kantenkontraktion, bei der in jeder Aktualisierungsiteration (Algorithmus 2) die Kontraktionsmenge S durch ein maximales Matching (Definition 6(b)) bestimmt wird. Die Berechnung des maximalen Matchings erfolgt hierbei gemäß der *One-Phase-Handshaking*-Variante des *Luby-Jones*-

4 Experimente

Handshaking-Algorithmus (Cohen & Castonguay, 2012). Insbesondere bei großen Graphen zeigt sich dabei ein typisches Verhalten: Während die Kardinalität von S zu Beginn kontinuierlich abnimmt, bildet sich in späteren Iterationen häufig ein Plateau, bei dem die Anzahl der gefundenen Matchings über viele Iterationen nahezu konstant bleibt. Dieses Stagnationsverhalten führt zu einer signifikanten Beeinträchtigung der Effizienz des Verfahrens. Daher wird, wie in der primalen Strategie von Abbas und Swo-boda (2022), eine Schranke eingeführt: Sobald die Kardinalität der Kontraktionsmenge diese unterschreitet, wird für die verbleibenden Iterationen die Kontraktionsmenge S durch die Berechnung eines maximalen, konfliktfreien Spannwaldes bestimmt. Dabei ist der Hyperparameter τ zu berücksichtigen, sodass die Schranke

$$|S| < |V| \cdot \tau \cdot 0.1$$

gilt.

PECF: Diese Methode beschreibt eine parallele Kantenkontraktion, bei der in jeder Aktualisierungsiteration (Algorithmus 2) die Kontraktionsmenge S durch einen maximalen, konfliktfreien Spannwald bestimmt wird. Hierbei wird die Implementierung von Algorithmus 3 verwendet.

Algorithmus	τ	Connectomics-SP				Cityscapes	
		Small (3)		Medium (3)		(59)	
		$C(\times 10^5)$	$t(s)$	$C(\times 10^5)$	$t(s)$	$C(\times 10^6)$	$t(s)$
GAEC	/	-1.794	1.8	-9.224	17.6	-1.827	41.4
PECM	1.00	-1.771	2.7	-9.139	24.7	-1.834	86.8
	0.30	-1.772	2.8	-9.140	25.9	-1.834	99.5
	0.20	-1.772	3.0	-9.149	28.1	-1.822	113.7
	0.10	-1.776	3.6	-9.160	33.6	-1.810	152.6
	0.05	-1.778	5.0	-9.169	45.3	-1.808	230.3
	0.01	-1.785	15.5	-9.190	138.0	-1.774	834.0
PECF	1.00	-1.784	2.5	-9.186	24.2	-1.841	50.9
	0.30	-1.791	2.7	-9.214	26.1	-1.844	58.3
	0.20	-1.793	2.9	-9.219	28.3	-1.844	62.5
	0.10	-1.793	3.6	-9.223	34.1	-1.843	78.3
	0.05	-1.793	5.0	-9.224	45.6	-1.848	106.8
	0.01	-1.794	14.5	-9.225	138.2	-1.848	340.4

Tabelle 4.1: Ergebnisse auf Connectomics- und Cityscapes-Instanzen mit verschiedenen τ -Werten

4 Experimente

Algorithmus	τ	ABR		MCF		Cluster Editing	
		(26)		(36)		(20)	
		$C(\times 10^4)$	$t(s)$	$C(\times 10^3)$	$t(s)$	$C(\times 10^2)$	$t(s)$
GAEC	/	-7.670	0.03	-6.260	0.01	-6.10	0.002
PECM	1.00	-7.608	0.04	-6.249	0.02	-5.86	0.002
	0.30	-7.623	0.04	-6.247	0.02	-5.90	0.002
	0.20	-7.637	0.05	-6.247	0.02	-5.95	0.003
	0.10	-7.657	0.06	-6.247	0.02	-6.02	0.003
	0.05	-7.659	0.08	-6.248	0.02	-6.05	0.004
	0.01	-7.665	0.25	-6.259	0.06	-6.10	0.008
PECF	1.00	-7.647	0.05	-6.226	0.02	-5.96	0.003
	0.30	-7.667	0.05	-6.240	0.02	-5.98	0.003
	0.20	-7.651	0.06	-6.247	0.02	-6.03	0.004
	0.10	-7.642	0.07	-6.256	0.03	-6.07	0.004
	0.05	-7.666	0.10	-6.260	0.04	-6.11	0.006
	0.01	-7.671	0.39	-6.258	0.12	-6.10	0.015

Tabelle 4.2: Ergebnisse auf MCR-, ABR- und Cluster-Edit-Instanzen mit verschiedenen τ -Werten

Algorithmus	τ	Random		Equicut		Correlation	
		(93)		(11)		(30)	
		$C(\times 10^6)$	$t(s)$	$C(\times 10^2)$	$t(s)$	$C(\times 10^3)$	$t(s)$
GAEC	/	-1.220	0.71	-7.91	0.0007	-3.206	0.002
PECM	1.00	-1.048	0.89	-7.43	0.0008	-2.805	0.002
	0.30	-1.045	1.00	-7.57	0.0008	-2.856	0.002
	0.20	-1.043	1.15	-7.66	0.0009	-2.891	0.002
	0.10	-1.047	1.61	-7.85	0.0010	-3.025	0.003
	0.05	-1.063	2.36	-7.99	0.0015	-3.207	0.004
	0.01	-1.159	6.64	-7.91	0.0023	-3.206	0.008
PECF	1.00	-1.091	0.98	-7.66	0.0010	-3.007	0.003
	0.30	-1.135	1.10	-7.73	0.0010	-3.104	0.003
	0.20	-1.154	1.21	-7.70	0.0017	-3.197	0.003
	0.10	-1.187	1.43	-7.88	0.0018	-3.231	0.004
	0.05	-1.212	1.92	-7.97	0.0021	-3.265	0.005
	0.01	-1.244	5.33	-7.91	0.0037	-3.206	0.012

Tabelle 4.3: Ergebnisse auf Random-, Equicut- und Correlation-Instanzen mit verschiedenen τ -Werten

4 Experimente

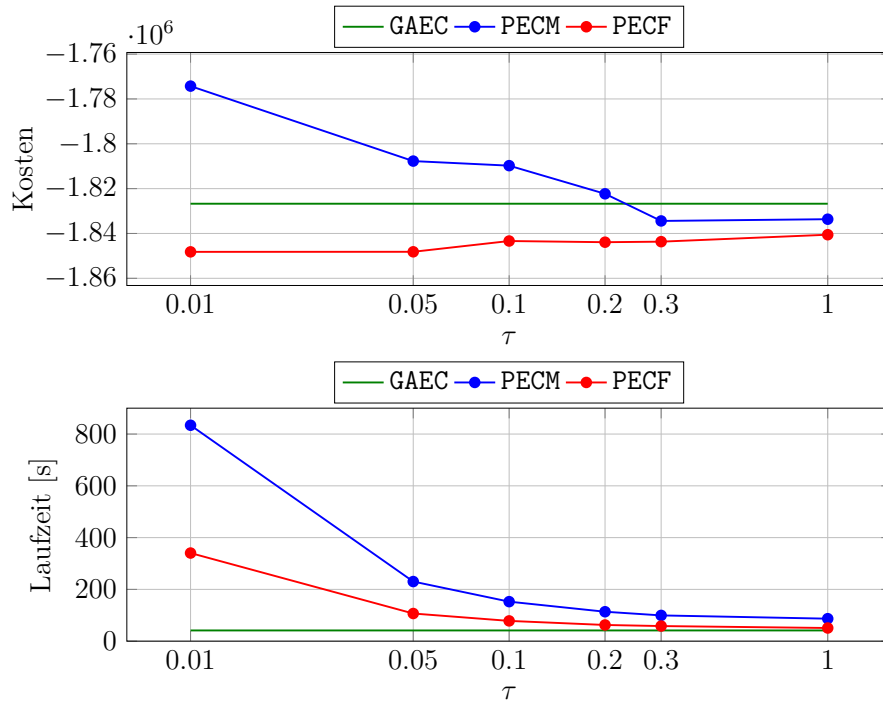


Abbildung 4.1: Vergleich von Zielfunktion (oben) und Laufzeit (unten) der Algorithmen bei variierendem τ auf dem *Cityscapes*-Datensatz.

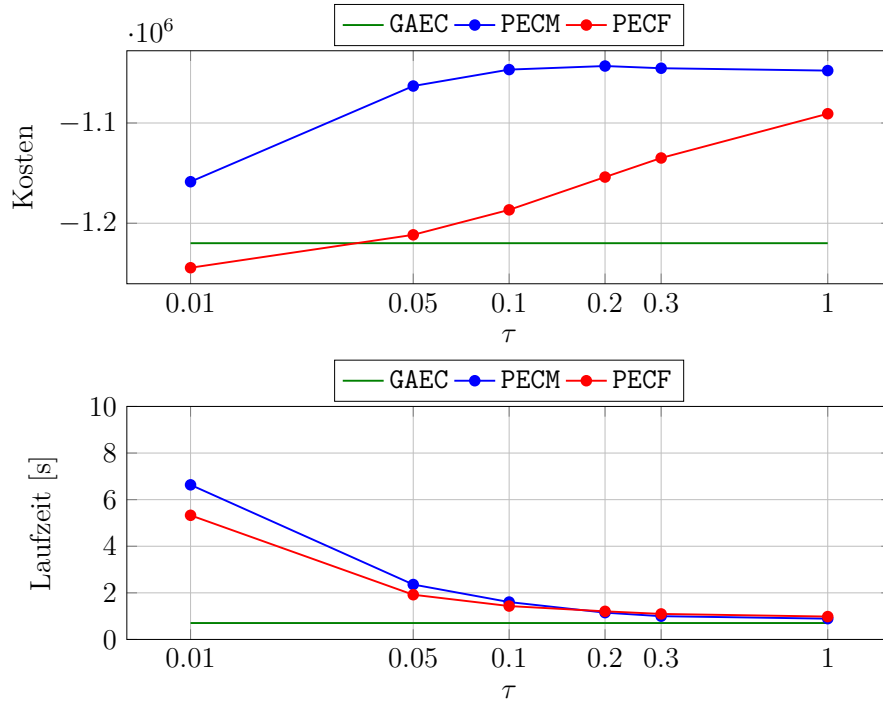


Abbildung 4.2: Vergleich von Zielfunktion (oben) und Laufzeit (unten) der Algorithmen bei variierendem τ auf dem *Random*-Datensatz.

4 Experimente

Die Ergebnisse für alle Datensätze sind in den Tabellen 4.1, 4.2 und 4.3 zusammengefasst. Kosten C und Laufzeit t sind als Mittelwert über alle Instanzen eines Datensatzes angegeben, wobei das jeweils beste Ergebnis hervorgehoben ist. Da alle Datensätze aus CP-Lib (Sørensen & Letchford, 2024) ausschließlich ganzzahlige Kosten enthalten, treten vermehrt Kanten mit identischen Kosten auf. Um konsistente Entscheidungen der betrachteten Algorithmen in diesen Fällen zu gewährleisten, wurde den entsprechenden Graphen ein Rauschen hinzugefügt.

Insgesamt zeigt sich ein Trend zunehmender Lösungsqualität bei fallendem τ für die betrachteten Datensätze. Besonders beim PECF ist dieses Verhalten deutlich erkennbar, dessen beste Ergebnisse konsistent für $\tau = 0.01$ bzw. $\tau = 0.05$ erzielt werden. Sie sind jeweils mindestens so niedrig wie die des GAEC, in den meisten Fällen niedriger. Dies zeigt sich insbesondere am *Random*-Datensatz, bei dem der PECF mit abnehmendem τ schrittweise niedrigere Zielfunktionswerte liefert und schließlich die des GAEC unterschreitet (siehe Abbildung 4.2). Für den *Cityscapes*-Datensatz erzielen beide Algorithmen bereits für $\tau = 1$ niedrigere Zielfunktionswerte als GAEC. Abbildung 4.1 illustriert den Einfluss des Hyperparameters τ auf PECM und PECF im Vergleich zu GAEC für diesen Datensatz. Auffällig zeigt sich hierbei das Verhalten des PECM: Während die Zielfunktionswerte des PECF bei abnehmendem τ weiter sinken, steigen sie beim PECM für $\tau \leq 0.3$ schrittweise an. Dies deutet darauf hin, dass das Matching-Verfahren bei den gegebenen Graph-Topologien des *Cityscapes*-Datensatzes für niedrige τ -Werte suboptimale Kontraktionsmengen erzeugt, was zu höheren Zielfunktionswerten führt. Bei den weiteren Datensätzen folgt PECM zwar ebenfalls dem Trend sinkender Zielfunktionswerte mit abnehmendem τ , erreicht jedoch häufig nicht die Lösungsqualität von GAEC oder PECF.

Für Graphen $G = (V, E)$ mit $|V| \leq 100$, wie den Instanzen des *Cluster-Editing*-, *Equicut*- und *Correlation*-Datensatzes, spezialisieren sich PECM und PECF bei $\tau = 0.01$ zum GAEC und erzielen entsprechend identische Ergebnisse.

Zudem zeigt sich für alle Datensätze ein Anstieg der durchschnittlichen Laufzeit beider Algorithmen bei fallendem τ . Bei den großen Instanzen des *Cityscapes*-Datensatzes wird dieser Anstieg besonders deutlich. Ursache dafür ist die in Kapitel 3.2.3 erläuterte Zunahme der erforderlichen Aktualisierungssiterationen. Besonders betroffen ist PECM, da hierbei stets das vollständige Matching berechnet werden muss. Bei PECF ist der Anstieg ebenfalls erkennbar, da die genutzte Implementierung in jeder Aktualisierungssiteration die Kanten aus dem aktuellen Graphen extrahiert und die Menge der positiven Kanten sortiert.

5 Fazit

In dieser Arbeit wurden heuristische Verfahren mittels Kantenkontraktionen zur Lösung des Multicut-Problems untersucht, einem bedeutenden kombinatorischen Optimierungsproblem im Bereich des maschinellen Lernens und Computer Vision. Während der von Keuper et al. (2015) vorgestellte GAEC-Algorithmus in jeder Iteration genau eine Kante mit maximalen Kosten kontrahiert, wird in der von Abbas und Swoboda (2022) vorgeschlagenen Erweiterung jeweils die Kontraktion einer Menge von Kanten vorgenommen. Zur Bestimmung dieser Kontraktionsmenge werden zwei Heuristiken eingesetzt: ein maximales Matching sowie ein maximaler, konfliktfreier Spannwald. Der Schwerpunkt dieser Arbeit lag auf dem qualitativen Vergleich der erzielten Ergebnisse dieser Heuristiken mit denen des GAEC-Algorithmus. Darüber hinaus wurde ein Hyperparameter $\tau \in [0, 1]$ eingeführt, um die maximal zulässige Kardinalität der Kontraktionsmenge zu regulieren und dessen Einfluss auf die Zielfunktion zu analysieren.

Für die in Kapitel 4 betrachteten Datensätze zeigt sich bei beiden Heuristiken ein Trend sinkender Zielfunktionswerte mit abnehmendem τ . Insbesondere die ausschließliche Verwendung maximaler, konfliktfreier Spannwälder als Kontraktionsmenge resultiert bei niedrigen τ -Werten in den konsistent besten Ergebnissen dieser Heuristik. Dabei sind die Zielfunktionswerte für $\tau = 0.01$ bzw. $\tau = 0.05$ stets mindestens so niedrig wie die des GAEC, in den meisten Fällen sogar niedriger. Dies spricht dafür, dass mit dieser Strategie global suboptimale lokale Optima, die der GAEC möglicherweise findet, vermieden werden können. Der auf maximalem Matching basierende Ansatz erreicht hingegen für die betrachteten Datensätze häufig nicht die Lösungsqualität der beiden anderen Verfahren.

Mit abnehmendem τ steigt die Anzahl erforderlicher Aktualisierungsiterationen, wodurch die mittlere Laufzeit beider Heuristiken zunimmt. Da der *Luby-Jones-Handshaking*-Algorithmus, wie in Kapitel 3.2.3 erläutert, stets das vollständige Matching berechnen muss, eignet er sich nicht optimal für eine CPU-basierte Implementierung in Verbindung mit dem Hyperparameter τ . Zudem besteht bei den genutzten Implementierungen beider Heuristiken weiterhin Optimierungsbedarf hinsichtlich ihrer Effizienz.

Die Experimente wurden auf Datensätze kleiner bis mittlerer Größe beschränkt, sodass die Übertragbarkeit der gewonnenen Ergebnisse auf sehr große Graphen noch zu prüfen ist.

Literaturverzeichnis

- Abbas, A. & Swoboda, P. (2021). Combinatorial optimization for panoptic segmentation: A fully differentiable approach. *Advances in Neural Information Processing Systems*, 34, 15635–15649.
- Abbas, A. & Swoboda, P. (2022). Rama: A rapid multicut algorithm on gpu. In *Proceedings of the ieee/cvf conference on computer vision and pattern recognition* (S. 8193–8202).
- Andres, B., Di Gregorio, S., Iрмаi, J. & Lange, J.-H. (2023). A polyhedral study of lifted multicut. *Discrete Optimization*, 47, 100757.
- Andres, B., Kappes, J. H., Beier, T., Köthe, U. & Hamprecht, F. A. (2011). Probabilistic image segmentation with closedness constraints. In *2011 international conference on computer vision* (S. 2611–2618).
- Andres, B., Kroeger, T., Briggman, K. L., Denk, W., Korogod, N., Knott, G., ... Hamprecht, F. A. (2012). Globally optimal closed-surface segmentation for connectomics. In *Computer vision–eccv 2012: 12th european conference on computer vision, florence, italy, october 7-13, 2012, proceedings, part iii 12* (S. 778–791).
- Bansal, N., Blum, A. & Chawla, S. (2004). Correlation clustering. *Machine learning*, 56, 89–113.
- Brunetta, L., Conforti, M. & Rinaldi, G. (1997). A branch-and-cut algorithm for the equicut problem. *Mathematical Programming*, 78 (2), 243–263.
- Chopra, S. & Rao, M. R. (1993). The partition problem. *Mathematical programming*, 59 (1), 87–115.
- Cohen, J. & Castonguay, P. (2012). Efficient graph matching and coloring on the gpu. In *Gpu technology conference* (S. 1–10).
- Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., ... Schiele, B. (2016, June). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the ieee conference on computer vision and pattern recognition (cvpr)*.
- CREMI-Challenge. (2016). *CREMI MICCAI Challenge on Circuit Reconstruction from Electron Microscopy Images*. <https://cremi.org>.
- Dahlhaus, E., Johnson, D. S., Papadimitriou, C. H., Seymour, P. D. & Yannakakis, M. (1994). The complexity of multiterminal cuts. *SIAM Journal on Computing*, 23 (4), 864–894.

- Demaine, E. D., Emanuel, D., Fiat, A. & Immorlica, N. (2006). Correlation clustering in general weighted graphs. *Theoretical Computer Science*, 361 (2-3), 172–187.
- Dua, D. & Graff, C. (2019). *UCI machine learning repository*. <http://archive.ics.uci.edu/ml/>. Irvine, CA.
- Grötschel, M. & Wakabayashi, Y. (1989). A cutting plane algorithm for a clustering problem. *Mathematical Programming*, 45, 59–96.
- Hornáková, A., Lange, J.-H. & Andres, B. (2017). Analysis and optimization of graph decompositions by lifted multicuts. In *International conference on machine learning* (S. 1539–1548).
- Hwu, W.-m. (2011). *Gpu computing gems jade edition* (Bd. 2). Elsevier.
- Jug, F., Levinkov, E., Blasse, C., Myers, E. W. & Andres, B. (2016). Moral lineage tracing. In *Proceedings of the ieee conference on computer vision and pattern recognition* (S. 5926–5935).
- Keuper, M., Levinkov, E., Bonneel, N., Lavoué, G., Brox, T. & Andres, B. (2015). Efficient decomposition of image and mesh graphs by lifted multicuts. In *Proceedings of the ieee international conference on computer vision* (S. 1751–1759).
- Keuper, M., Tang, S., Andres, B., Brox, T. & Schiele, B. (2018). Motion segmentation & multiple object tracking by correlation co-clustering. *IEEE transactions on pattern analysis and machine intelligence*, 42 (1), 140–153.
- Kim, S., Nowozin, S., Kohli, P. & Yoo, C. (2011). Higher-order correlation clustering for image segmentation. *Advances in neural information processing systems*, 24.
- Kirillov, A., Levinkov, E., Andres, B., Savchynskyy, B. & Rother, C. (2017). Instancecut: from edges to instances with multicut. In *Proceedings of the ieee conference on computer vision and pattern recognition* (S. 5008–5017).
- Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical society*, 7 (1), 48–50.
- Oosten, M., Rutten, J. H. & Spieksma, F. C. (2001). The clique partitioning problem: facets and patching facets. *Networks: An International Journal*, 38 (4), 209–226.
- Pape, C., Beier, T., Li, P., Jain, V., Bock, D. D. & Kreshuk, A. (2017). Solving large multicut problems for connectomics via domain decomposition. In *Proceedings of the ieee international conference on computer vision workshops* (S. 1–10).
- Simanchev, R. Y., Urazova, I. V. & Kochetov, Y. A. (2019). The branch and cut method for the clique partitioning problem. *Journal of Applied and Industrial Mathematics*, 13 (3), 539–556.
- Sørensen, M. M. & Letchford, A. N. (2024). Cp-lib: benchmark instances of the clique partitioning problem. *Mathematical Programming Computation*, 16 (1), 93–111.
- Swoboda, P., Andres, B., Hornakova, A., Bernard, F., Irmay, J., Roetzer, P., ... Abbas, A. (2022). Structured prediction problem archive. *arXiv preprint arXiv:2202.03574*.
- Wang, H., Alidaee, B., Glover, F. & Kochenberger, G. (2006). Solving group technology problems via clique partitioning. *International Journal of Flexible Manufacturing Systems*, 18 (2), 77–97.
- Wolf, S., Bailoni, A., Pape, C., Rahaman, N., Kreshuk, A., Köthe, U. & Hamprecht, F. A. (2021). The mutex watershed and its objective: Efficient, parameter-free graph

- partitioning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43 (10), 3724–3738. doi: 10.1109/TPAMI.2020.2980827
- Zheng, Z., Lauritzen, J. S., Perlman, E., Robinson, C. G., Nichols, M., Milkie, D., ... others (2018). A complete electron microscopy volume of the brain of adult *drosophila melanogaster*. *Cell*, 174 (3), 730–743.
- Zhou, Y., Hao, J.-K. & Goëffon, A. (2016). A three-phased local search approach for the clique partitioning problem. *Journal of Combinatorial Optimization*, 32 (2), 469–491.